KHUDA

Computer Vision

6주차

Computer Vision Task

트랙장 : 이승준

Contents

01 Localization

02 Object Detection

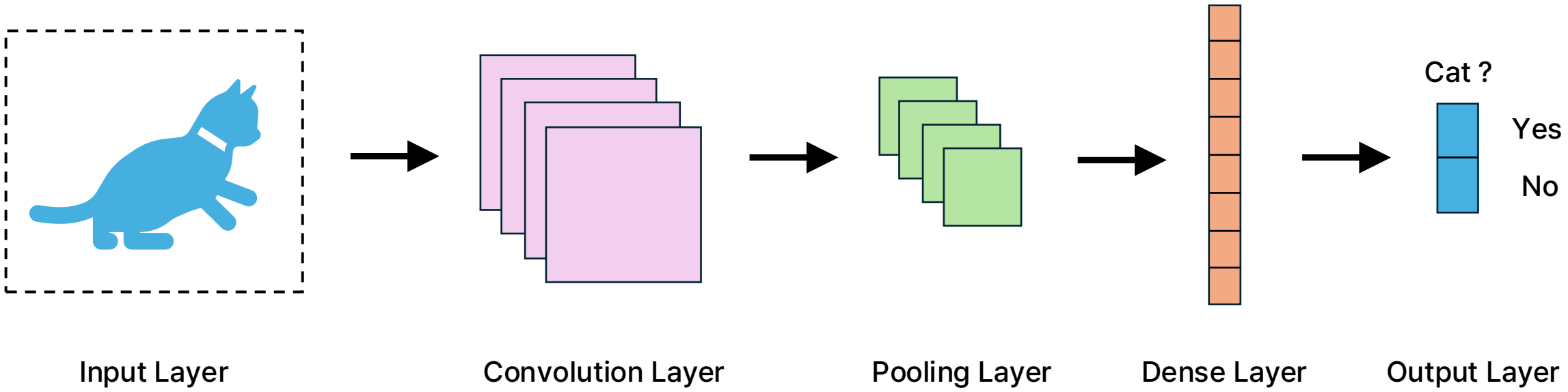
03 Semantic Segmentation

04 AutoEncoder

Localization

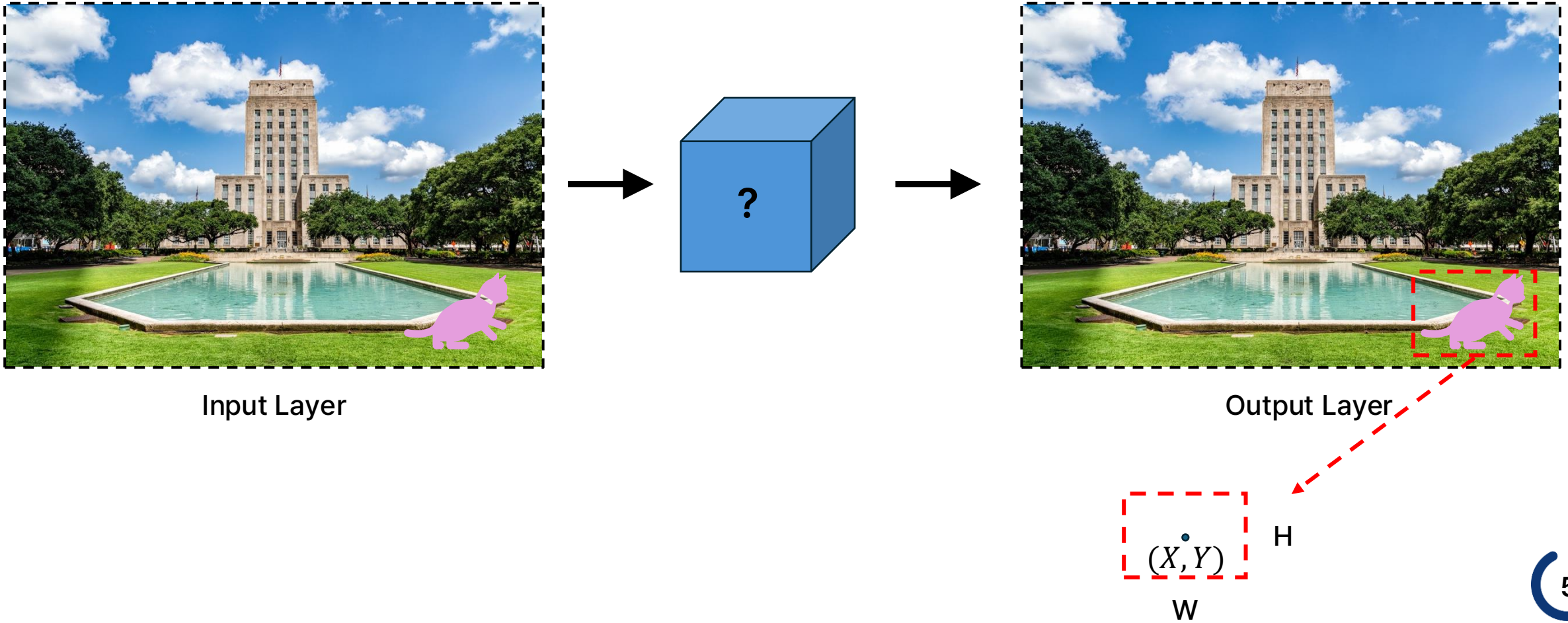
Classification

단순하게 사진 한장을 보고, 어떤 class인지 맞추는 것이 끝



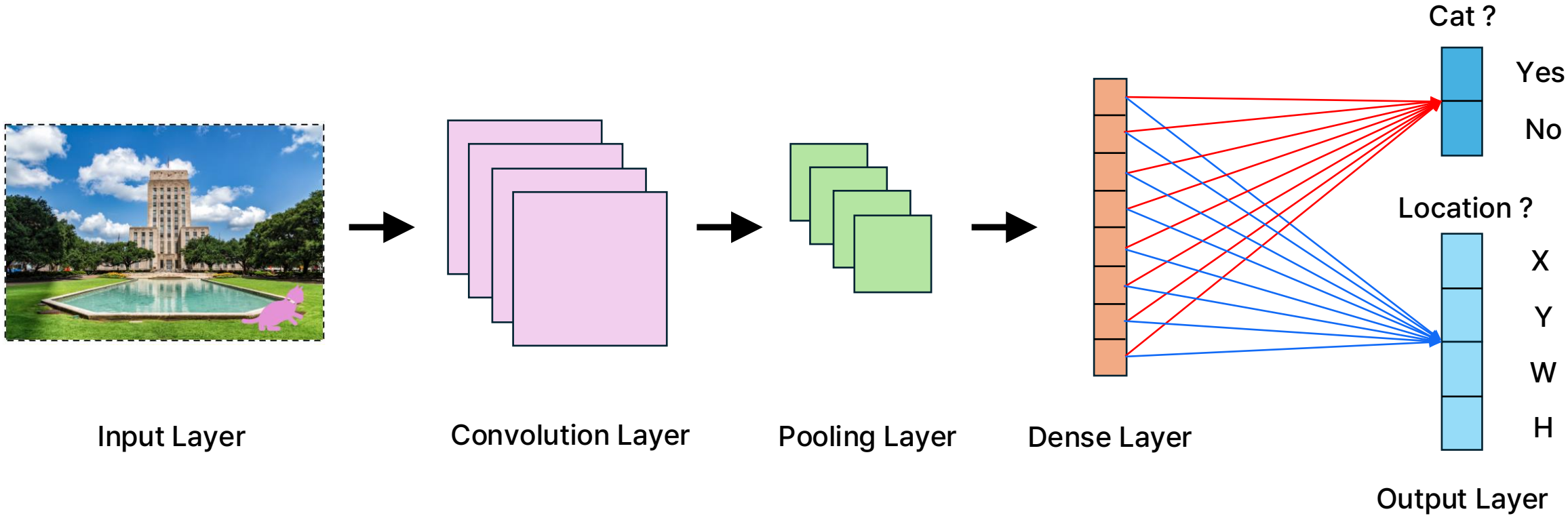
Localization

사진 속에서 고양이가 어디에 위치하는지가 중요하다면 ?



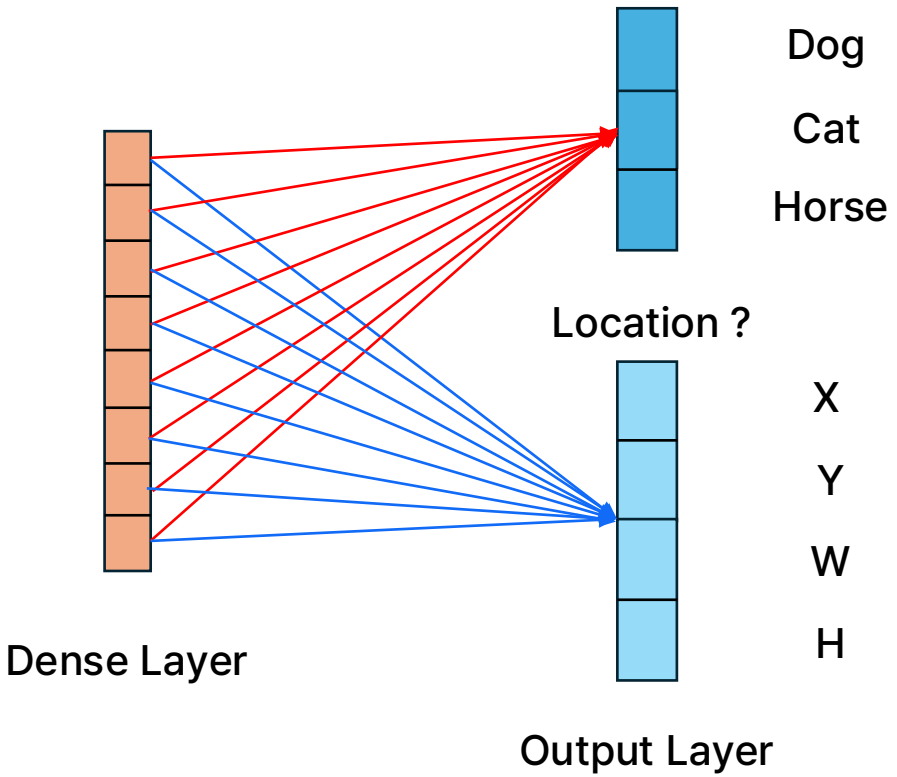
Classification + Localization

사진 속에서 고양이가 어디에 위치하는지가 중요하다면 ?



Classification + Localization

그럼 모델이 다른 종류의 output을 생성하는데, Loss는 어떻게 걸어주나요?

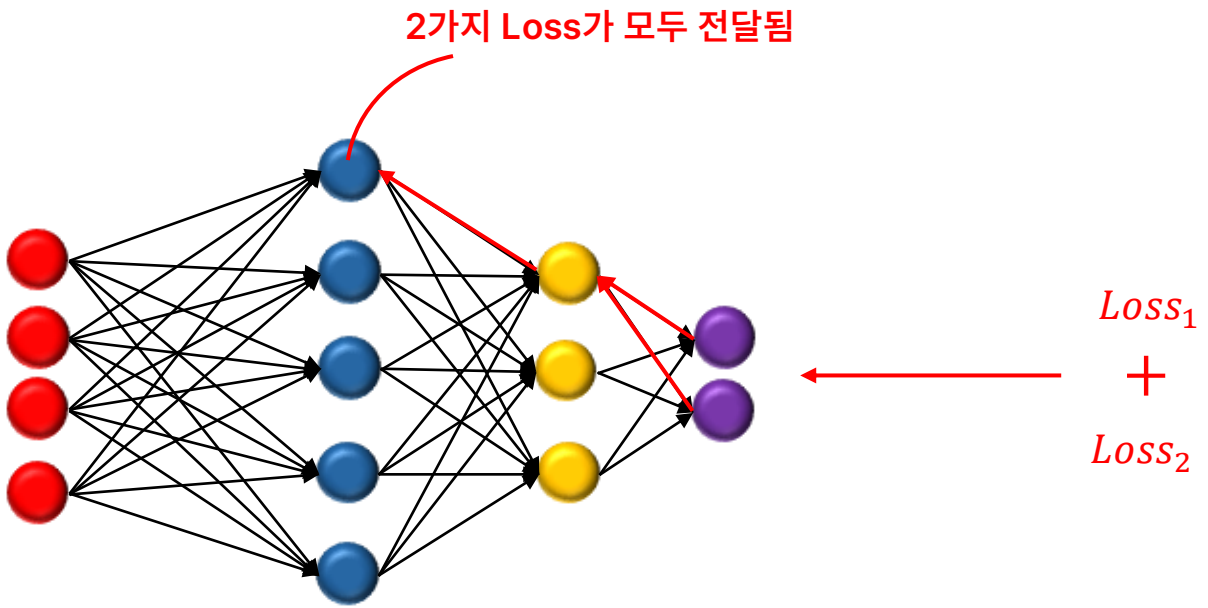


	Prediction		Ground Truth
Dog	0.06		0.00
Cat	0.91	$Loss_1$	1.00
Horse	0.03		0.00
X	190		192
Y	180	$Loss_2$	182
W	20		18
H	12		9

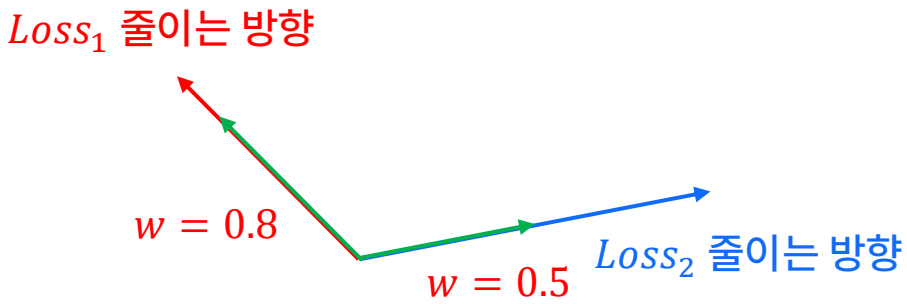
$+ \longrightarrow Total\ Loss$

Classification + Localization

Loss별로 가중치 변경에 대한 비율을 다르게 설정할 수 있음



하나의 노드에 여러 Loss가 들어오게 된다면,
그 중 어떤 Loss를 얼마만큼 반영해서 움직일 것인지를 설정할 수 있음

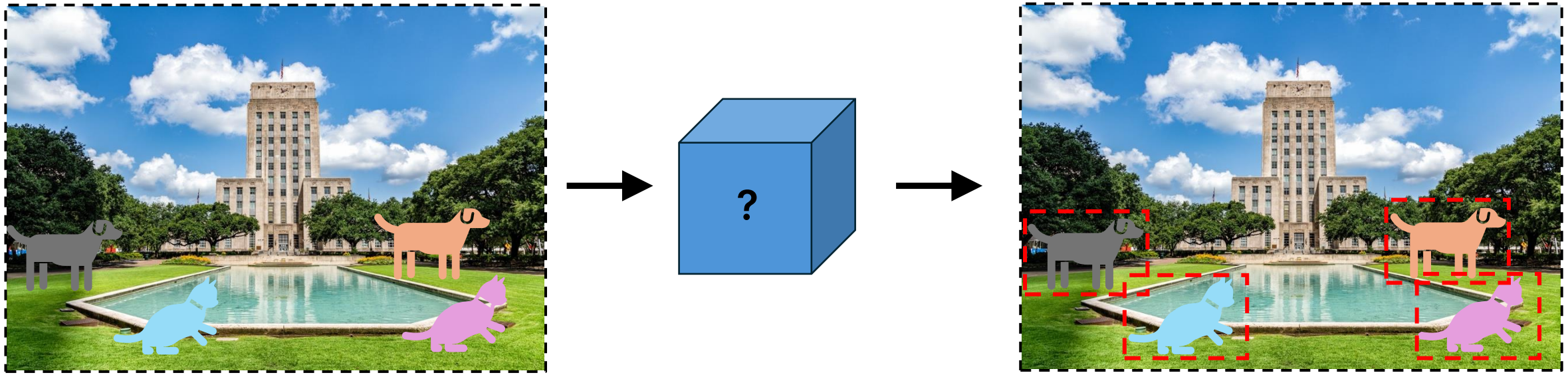



```
loss_cls=dict(type='FocalLoss', use_sigmoid=True, gamma=2, alpha=0.25, reduction='mean', loss_weight=1.0),
loss_bbox=dict(type='L1Loss', reduction='mean', loss_weight=0.25),
loss_heatmap=dict(type='GaussianFocalLoss', reduction='mean', loss_weight=1.0),
loss_seg=dict(type='CrossEntropyLoss', reduction='mean', loss_weight=0.005),
```

Object Detection

Object Detection

하나의 사진 속에 다양한 객체가 있다면 ?



Input Layer

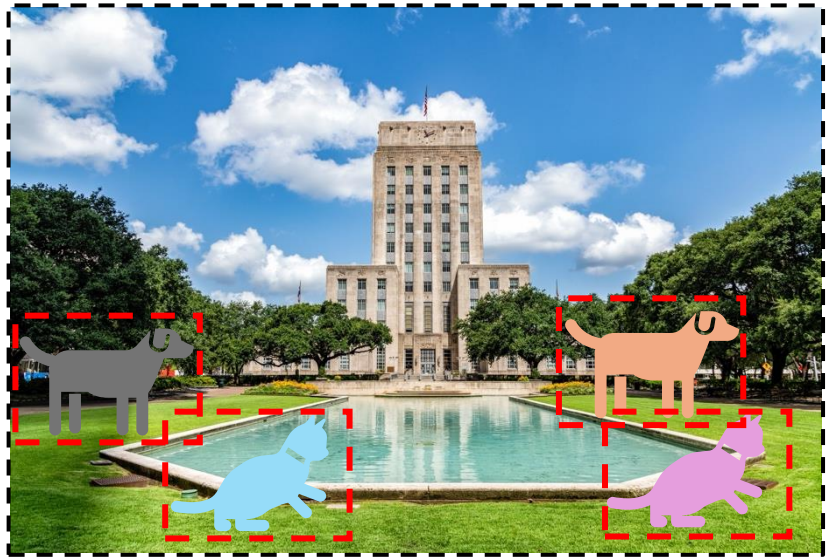
Output Layer

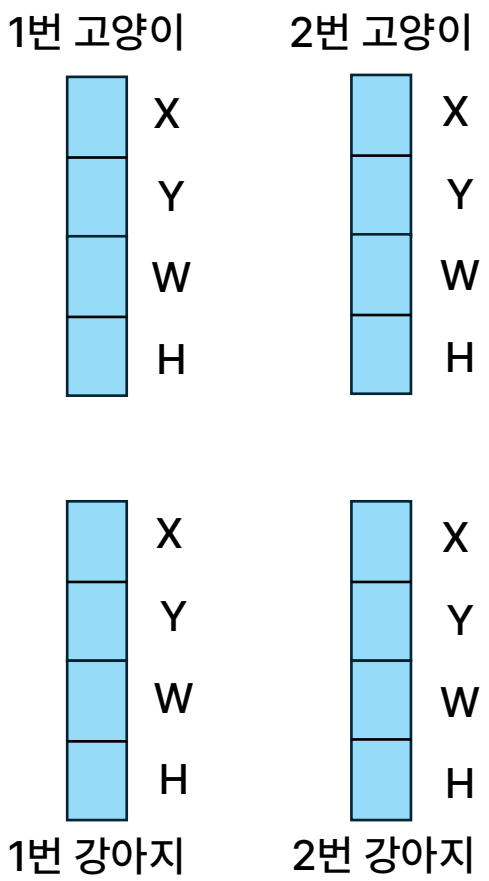
Object Detection as Regression

만약 하나의 그림 속에 여러 객체가 있다면, output node 개수를 어떻게 설정해야하는거죠?

객체가 몇 개 있을 줄 알고, 미리 output node 를 몇 개 만들어둬야하는걸까요 ?

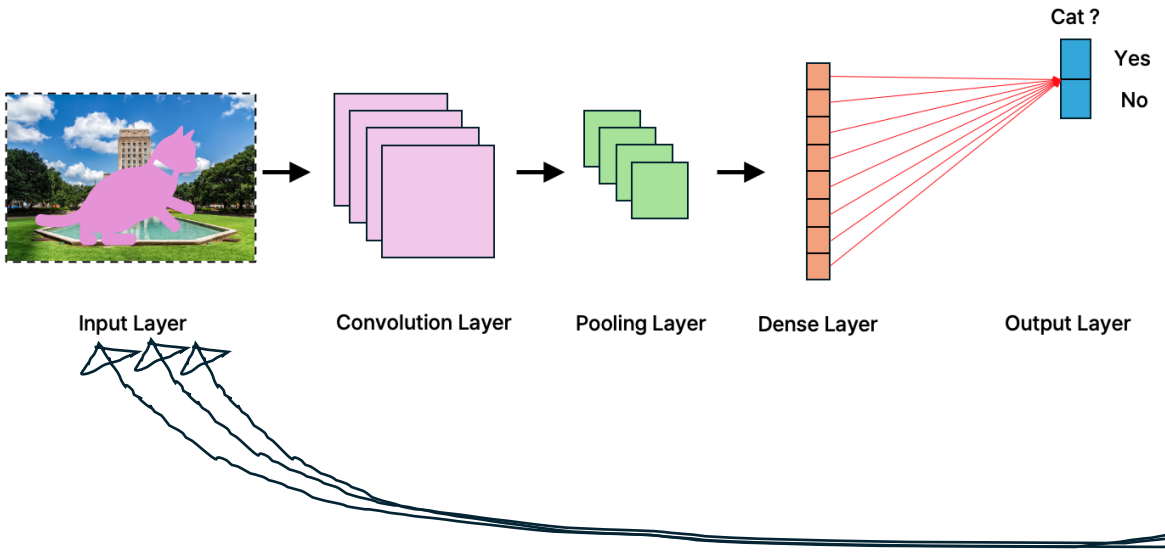
-> Object Detection을 단순히 Regression 문제로 생각하기엔 무리가 있다는 것

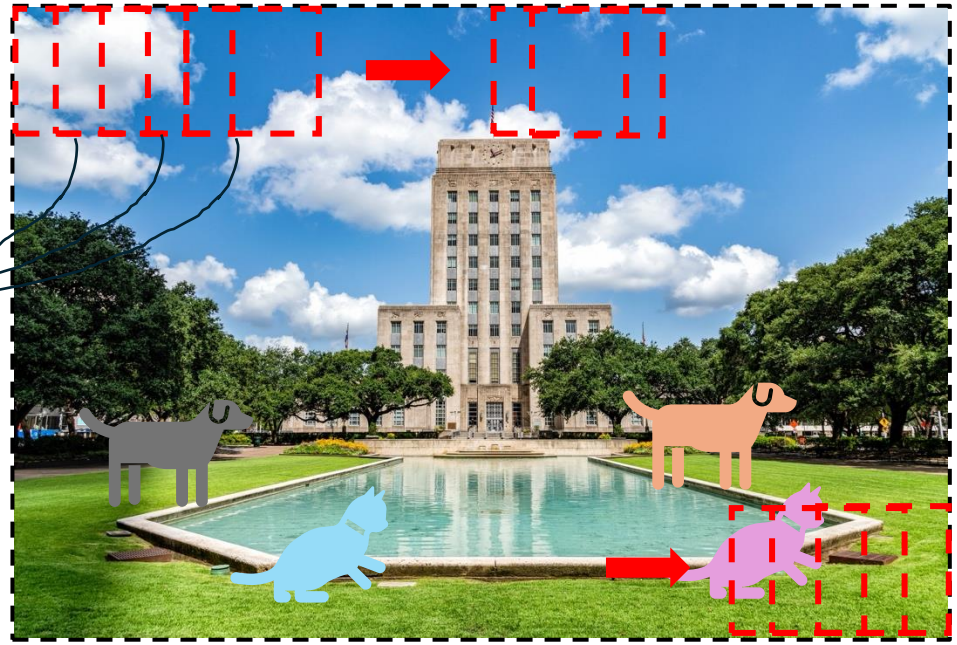




Object Detection with Sliding Window

그렇다면 Classification을 하는 모델을 하나만 사용해서, 이미지위에 window를 씌워서 각 window별로 하나씩 예측을 하자





문제점 1. window size는 어떻게 조절할건데 ?

문제점 2. sliding은 얼마만큼씩 움직일건데 ?

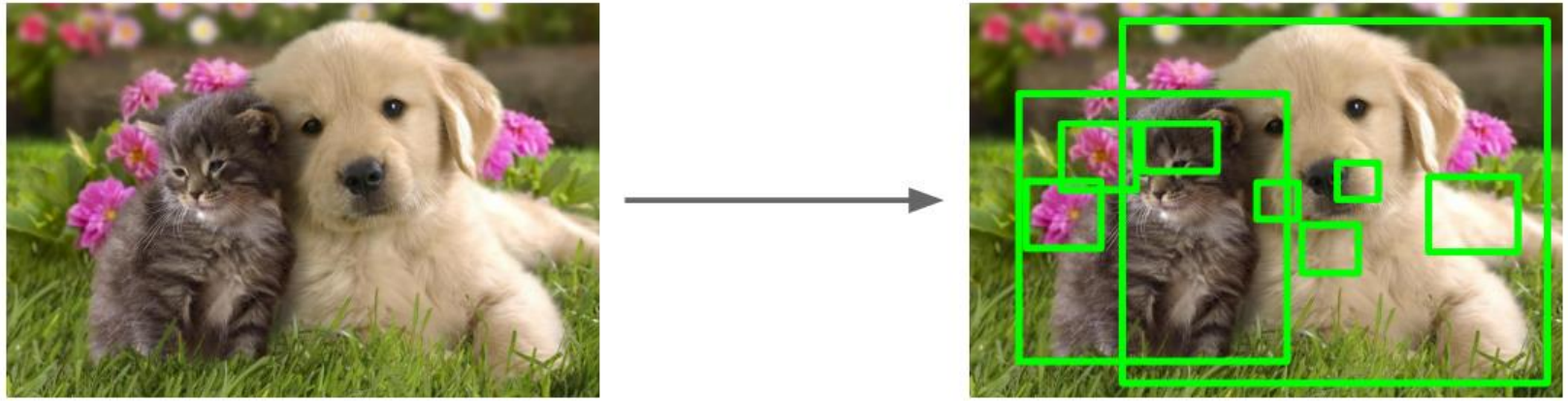
-> 이미지 속 객체가 너무 다양하기에 window를 어떻게 설계할지에 대한 문제가 생김

Object Detection with Region Proposal

특별한 알고리즘을 통해서, 이미지의 모든 범위를 다 확인하지 말고 관심 있게 봐야 할 부분들만 Pick up 하자

즉, window를 갈아끼우면서 찾는게 아니라, 확률이 높은 곳을 추천 받고

그 영역에 대해서만 classification을 수행하면서 bbox의 위치를 다듬자는 것



Selective Search Algorithm

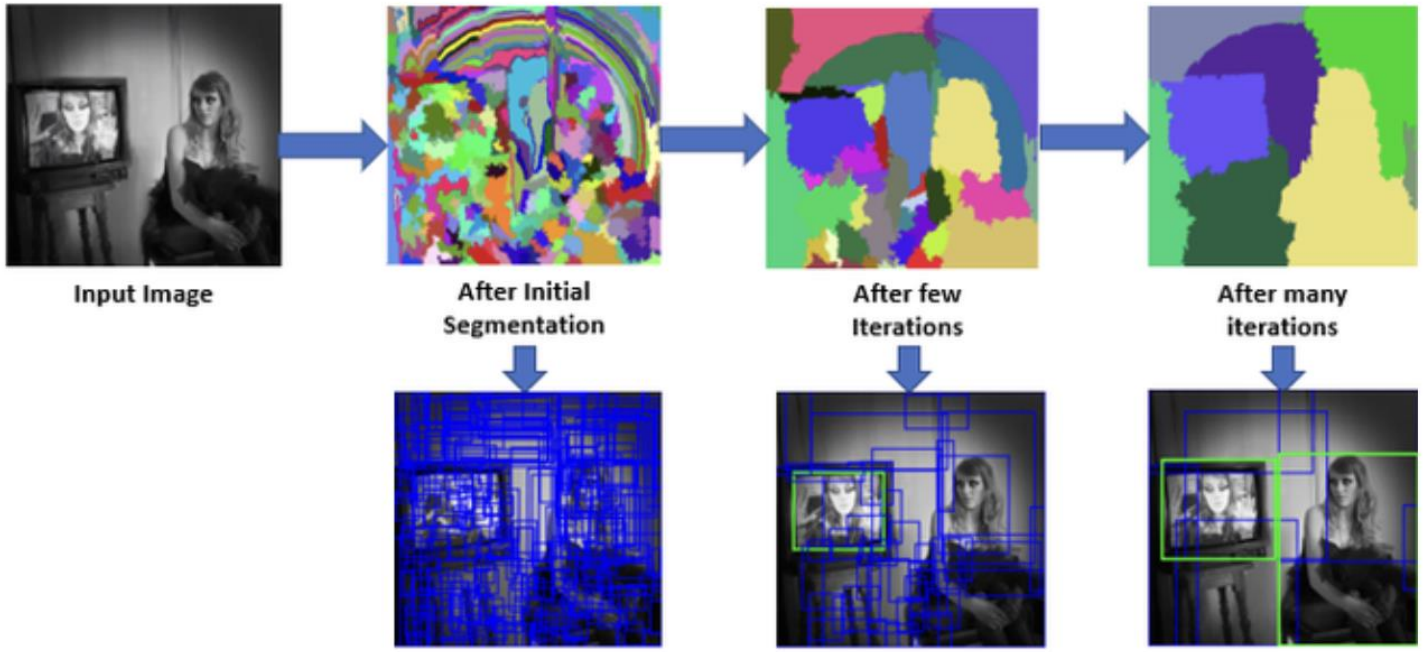
그럼 중요한 부분을 어떻게 선택할 것인지. 이게 핵심인 것 같은데 방법은 간단함

1. 원본 이미지에다 Segmentation을 진행함 (비슷해 보이는 녀석들끼리 같은 labeling을 주는 것)

2. 색상, 무늬, 크기, 형태 등등을 고려해서 합쳐진 영역을 사이의 유사도를 계산하고 한번 더 Segmentation을 진행

3. 이런 과정을 계속 반복해서 점점 유사도가 높은 영역끼리 합쳐져서, 객체가 있을 법한 부분에 대한 정보를 주게 됨

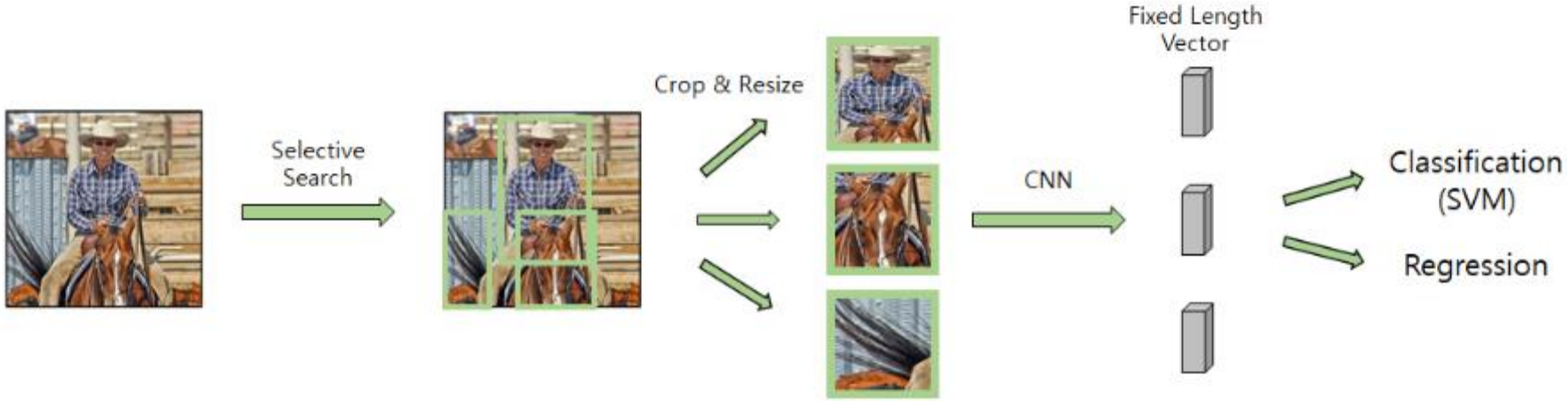
이렇게 찾아진 중요한 박스에서부터 시작해서 classification과 Bbox regression을 진행하는 것임



2 Stage Detector (R-CNN)

Object Detection은 크게 2가지 방법으로 나뉜다고 볼 수 있음

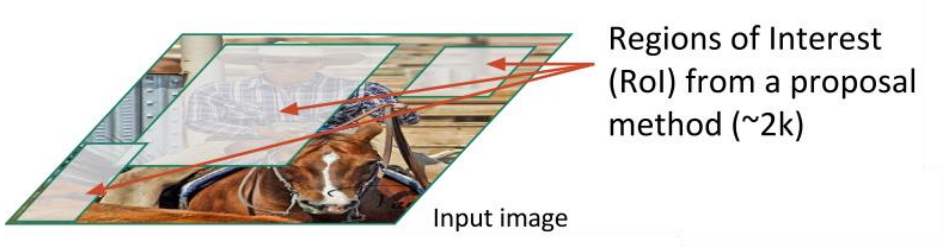
그 중 첫번째 방법이 2 Stage Detector인데, 간단히 말해 방금 본 방식으로 중요한 Region을 추천 받고, 각 Region 별로 Detection 하는 것



2 Stage Detector (R-CNN)

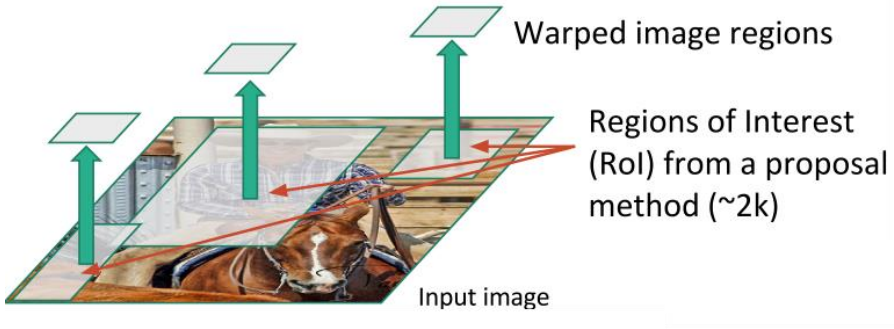
2 Stage Detector의 과정을 조금 더 자세히 들여다보면 다음과 같음

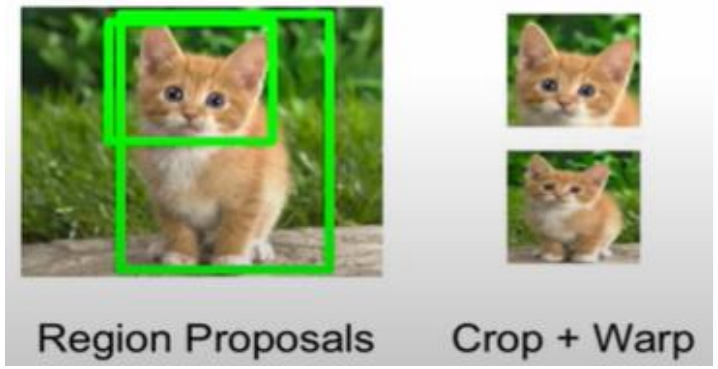
1. Selective Search 알고리즘을 통해 우리가 보고자 하는 Region Proposal을 얻음 (보통 2,000장 정도)



2. 추천 받은 Region의 size를 일치시켜줌

결국 추천 받은 모든 영역의 사진들이 하나의 모델 형태에 들어가는 것이니까, input size를 동일하게 만들어 줘야 함



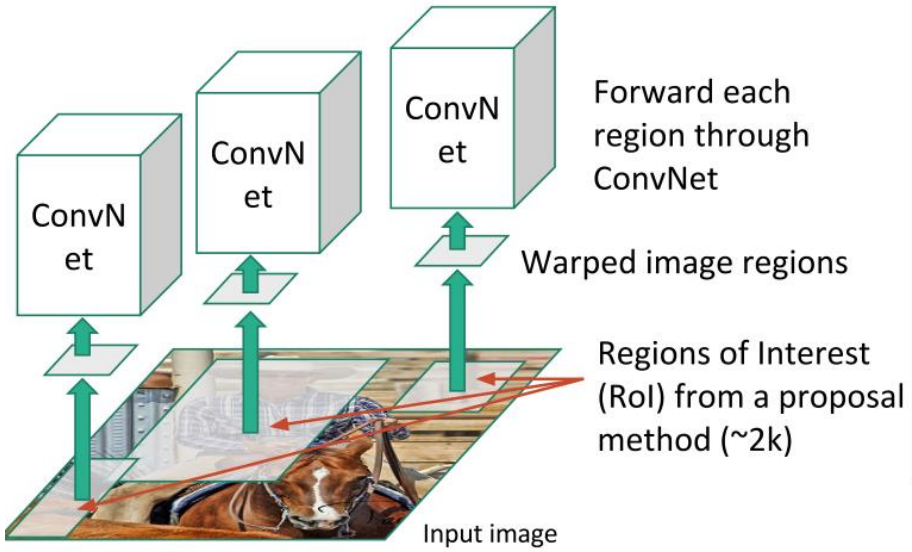


2 Stage Detector (R-CNN)

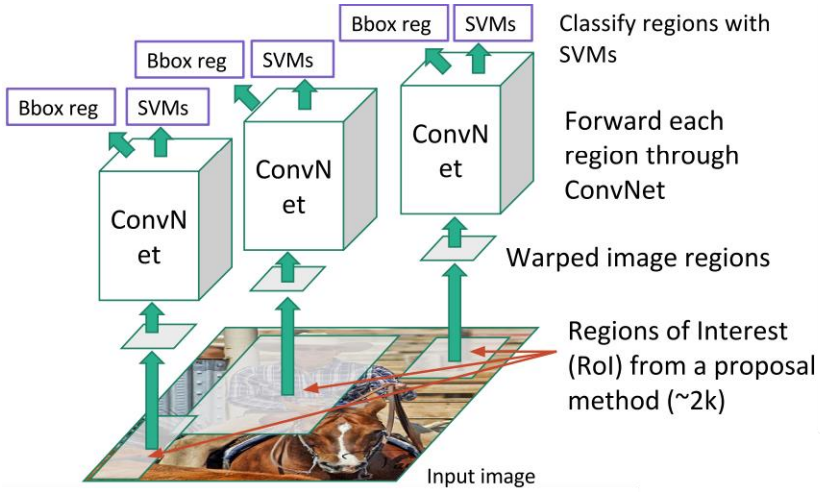
3. Warp 된 2,000장의 이미지를 각각 CNN에 넣고 feature map을 생성함

모든 이미지(영역)을 다 CNN에 넣으니까 계산이 너무 많아서 효율이 안좋음

아무리 영역을 좋게 추천 받아도 알고리즘의 한계로 불필요한 영역이 있음.



4. 2,000장에 대해서 추출된 Feature를 바탕으로 Fully Connected layer를 거쳐서 분류와 bbox 회귀를 진행함

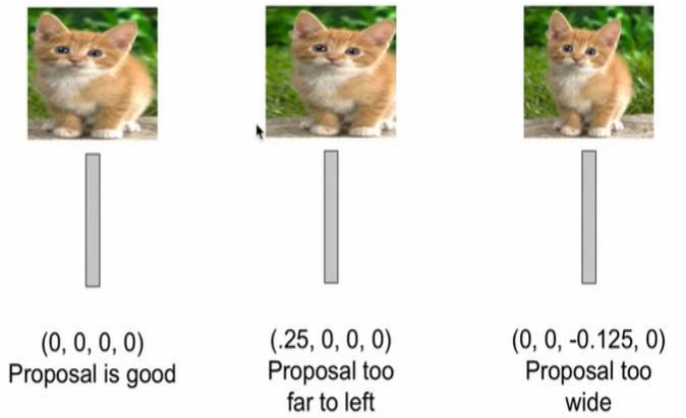


근데 애초에 박스가 추천되는데 왜 또 Bbox Regression을 하는 것이지 ?

2 Stage Detector (R-CNN)

추천 받은 영역이 오른쪽과 같이 완벽하지 못함

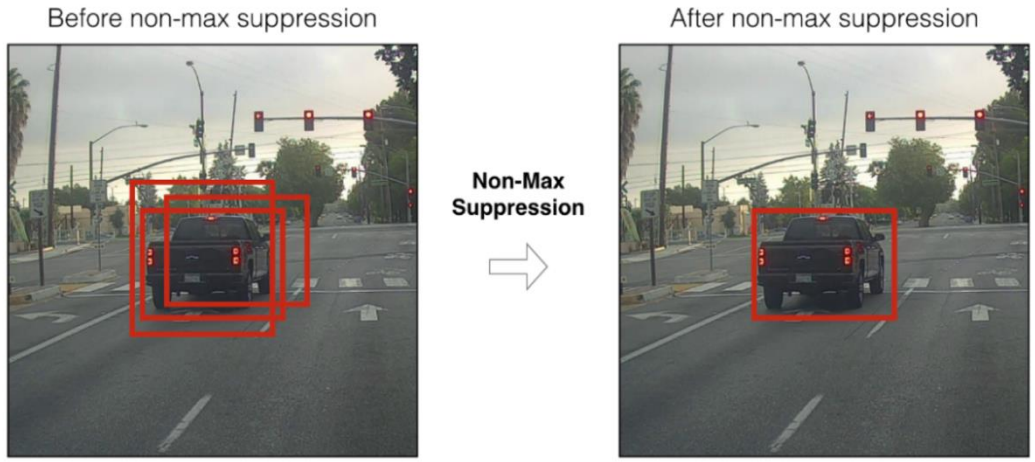
결국 학습이 아니라 Selective Search 알고리즘을 통해 얻은 것이기 때문에 퀄리티가 좋지 않기 때문에, 추가적인 Refinement가 필요함



5. 마지막으로 refinement까지 끝난 Box들에 NMS(Non-Maximum-Suppression)을 적용해서 객체마다 가장 좋은 box 하나를 남기게 됨

Bbox 마다 객체에 대한 확률값을 가지게 되는데 추천 받은 Bbox들이
겹치니까, 그 중에서 가장 확률이 높은 하나를 남기고 나머지를 제거하는 것

방법은 그냥 GT와의 IOU를 비교해서 하나씩 탈락시키면서 최종 선택

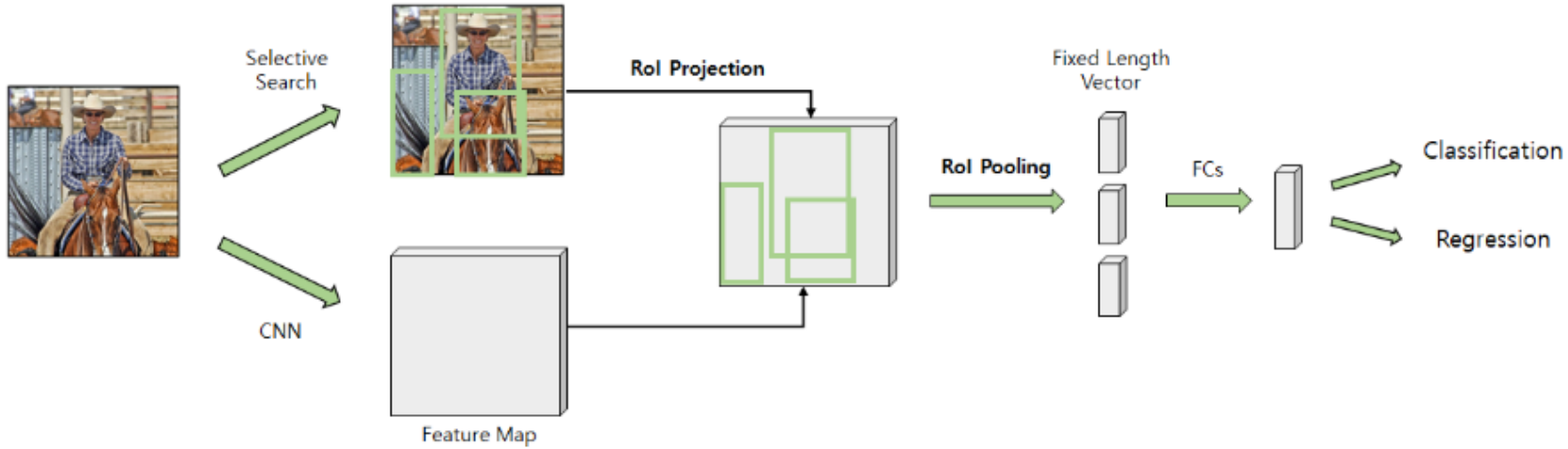


Fast R-CNN

앞에서 본 기본 R-CNN은 결국 CNN을 2000번 정도 돌려야 하니까 속도가 매우 느린 것임.

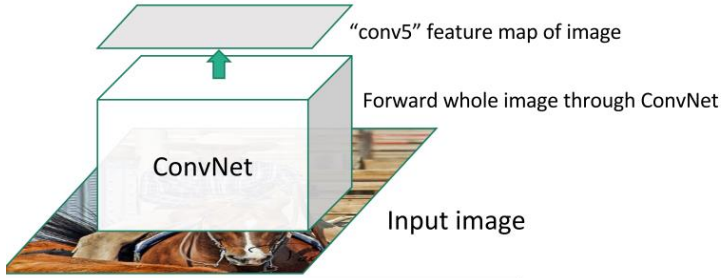
이를 개선하고자 Fast R-CNN이 등장했는데, 바뀐 것은 단순히 CNN을 이미지 당 1번만 돌리겠다는 것임.

즉, 원본 이미지에서 feature를 뽑고, 거기서 각 Region에 맞는 feature를 가져가서 FC Layer에 연결하는 것.

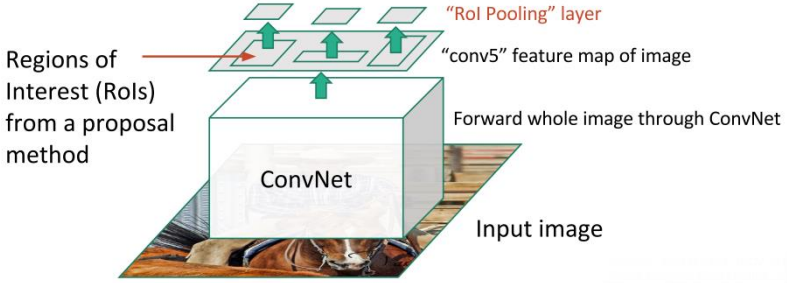


Fast R-CNN

1. 원본 이미지에 대해서 CNN을 1번 돌려서 1개의 Feature map을 생성함 (size가 아니라 모델이 한 번 돌아갔다는 의미)

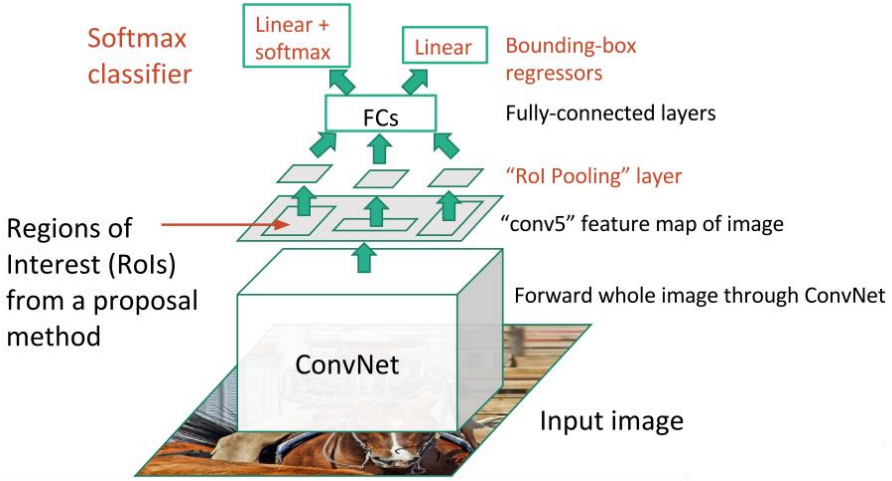


2. Selective Search 를 통해 뽑아낸 ROI(Region Of Interest)를 Feature map위에다가 씌워서 바로 Feature를 추출해 감



3. 이후 각각의 Region Feature에 대해서 FC Layer를 연결해서 Classification + Regression 수행

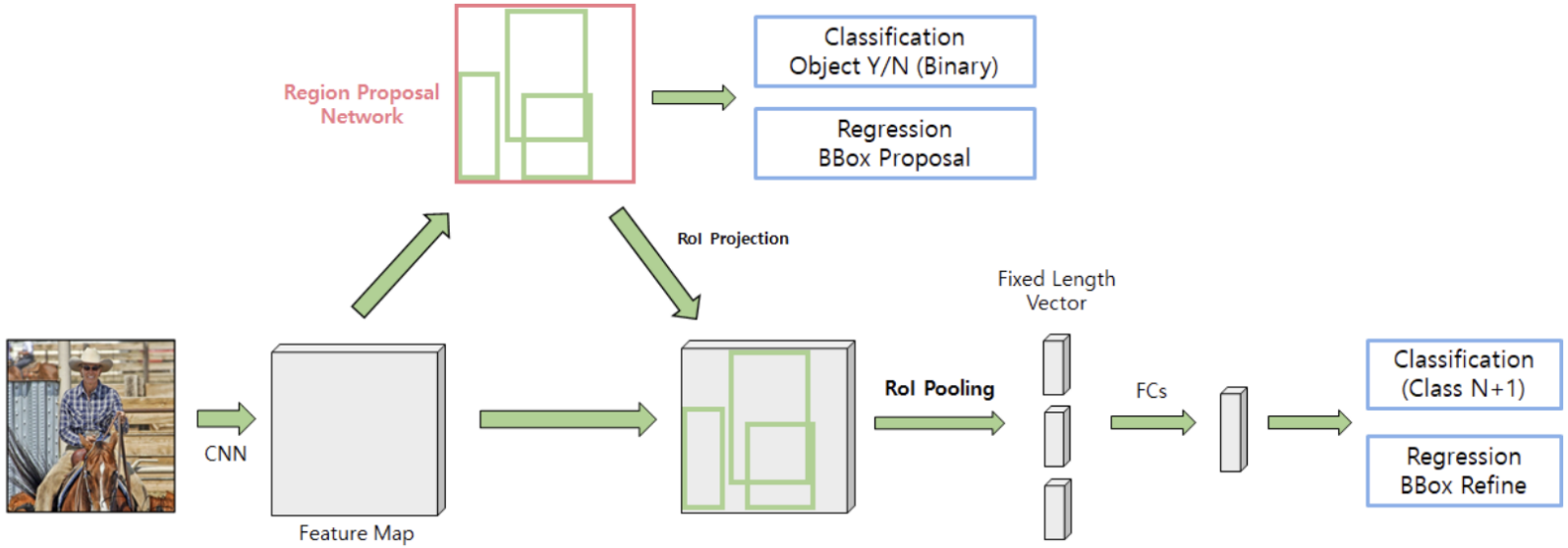
4. NMS를 통해 최종 Bbox들을 찾아냄



Faster R-CNN

그럼 Region Proposal 알고리즘을 대체할 수 있는 Network를 사용하면 되는거 아닌가 ?

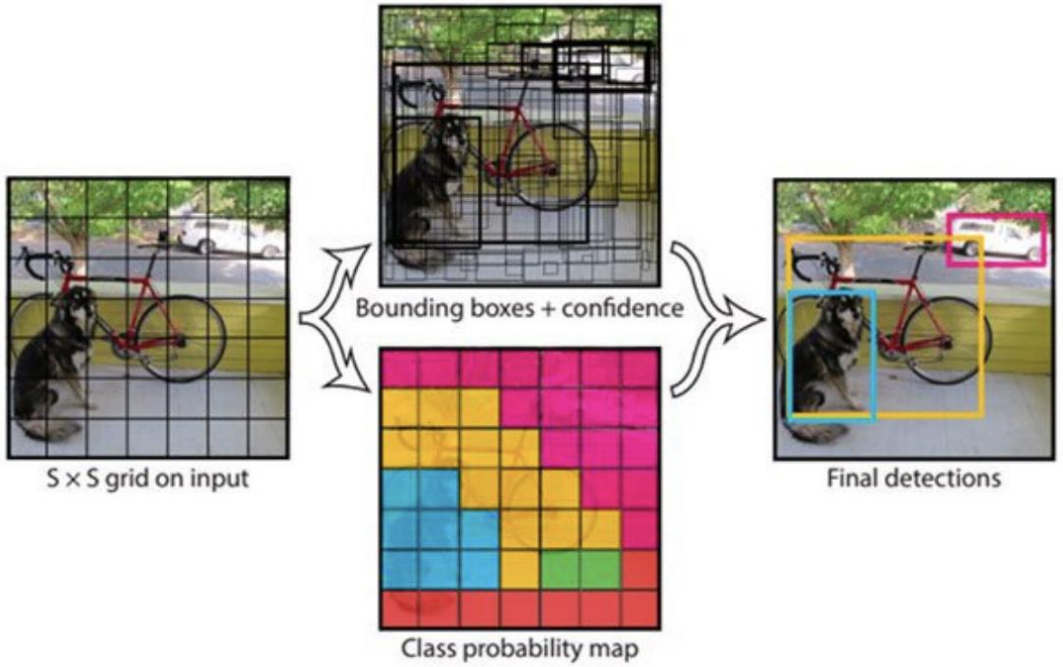
그래서 Faster R-CNN은 Region Proposal Network를 따로 만들어서 동작하도록 설계함



One-Stage Detector

One-Stage Detector는 결국 하나의 Regression 문제로 보겠다는 것인데, 대표적인 모델이 YOLO임

사진을 한번 보고, 객체의 위치와 객체의 class까지 모두 찾아내겠다는 것



Redmon, Joseph, et al. "You only look once: Unified, real-time object detection." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016.

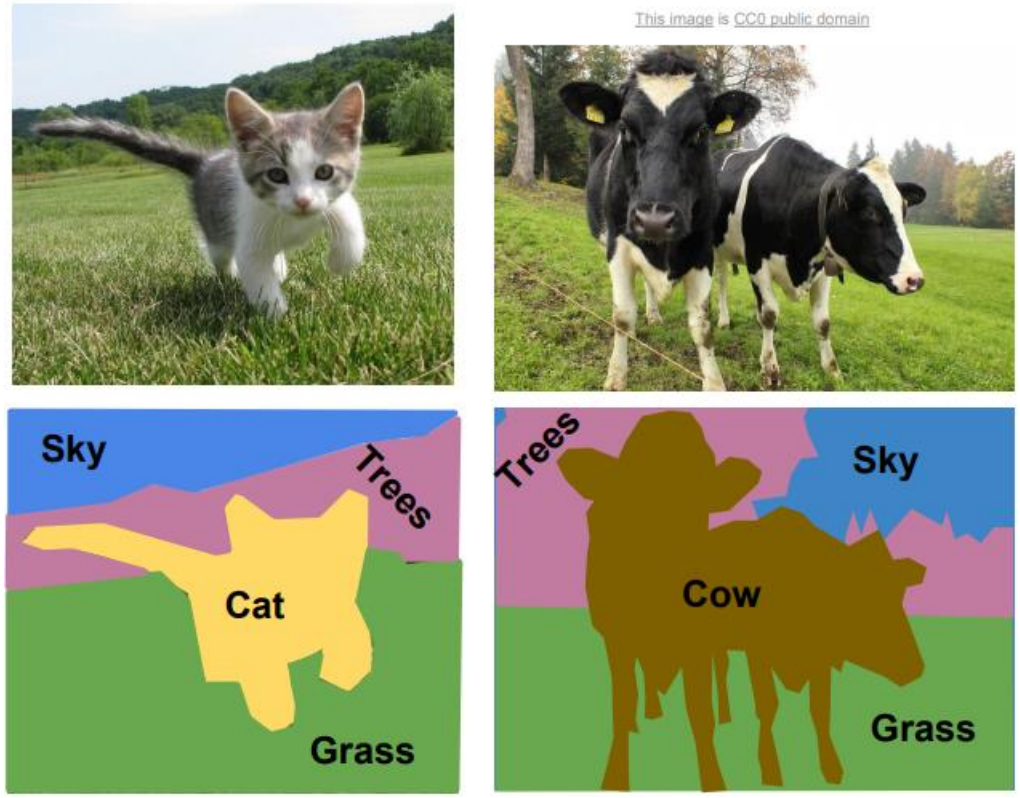
Semantic Segmetation

Semantic Segmentation

Semantic Segmentation은 Input 이미지에서 모든 픽셀에 대한 Label을 부여해주는 것임

즉, 이미지 한장을 보고 어떤 객체를 담고 있는지 Label을 부여하는 것이 아니라, 모든 픽셀에 대해서 이름표를 달아 준다는 것

이미지가 만약 224 x 224 (3 channel)이였다고 치면, 224 x 224 x 1 결과값을 얻게 되는 것임

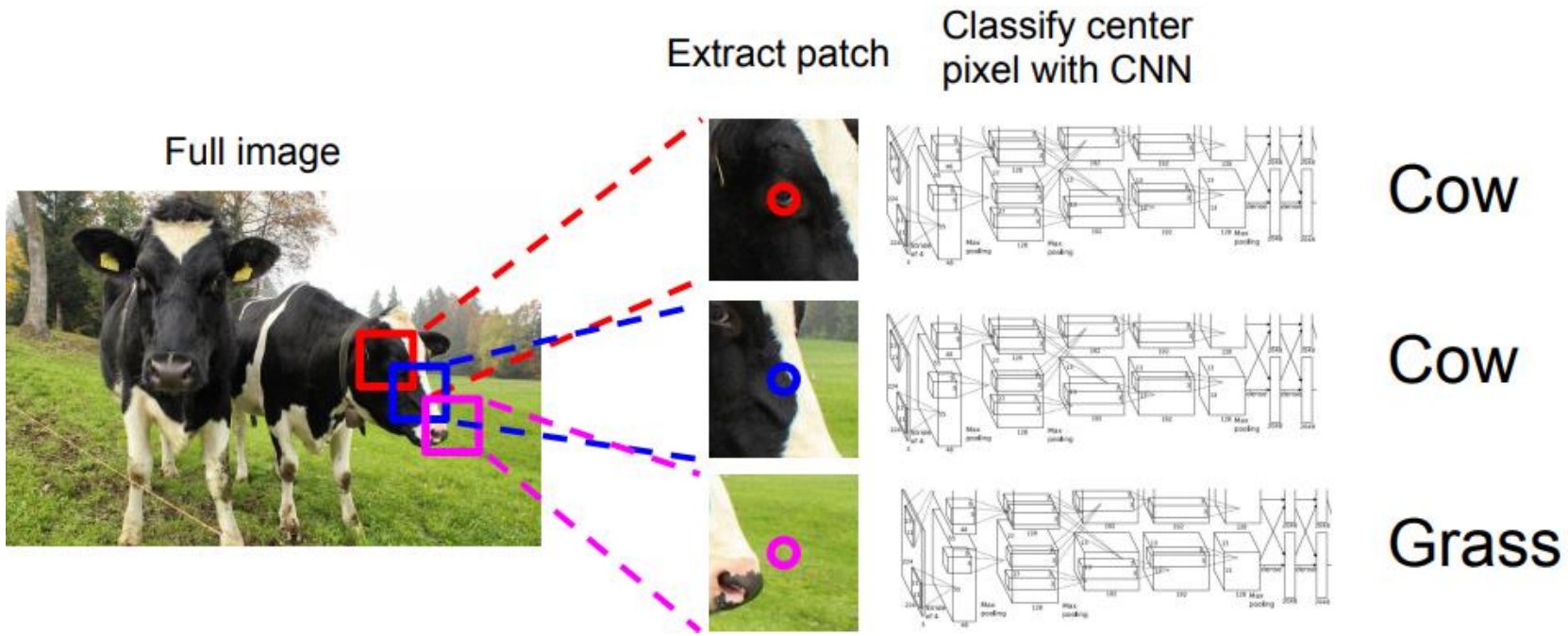


Segmentation with Sliding Window

이미지 한 장에 대해서 Semantic Segmentation을 수행하는 방법은 여러가지가 존재함

이미지에 window를 sliding하면서 모든 이미지를 돌면서, 각각의 window마다 Classification을 수행하여 중심 픽셀에 Label을 붙여주는 것

결론부터 말하자면 이 방식은 사용되지 않음. 각 이미지마다 CNN이 돌아가야 하고, window 크기에 대한 고려도 어려움



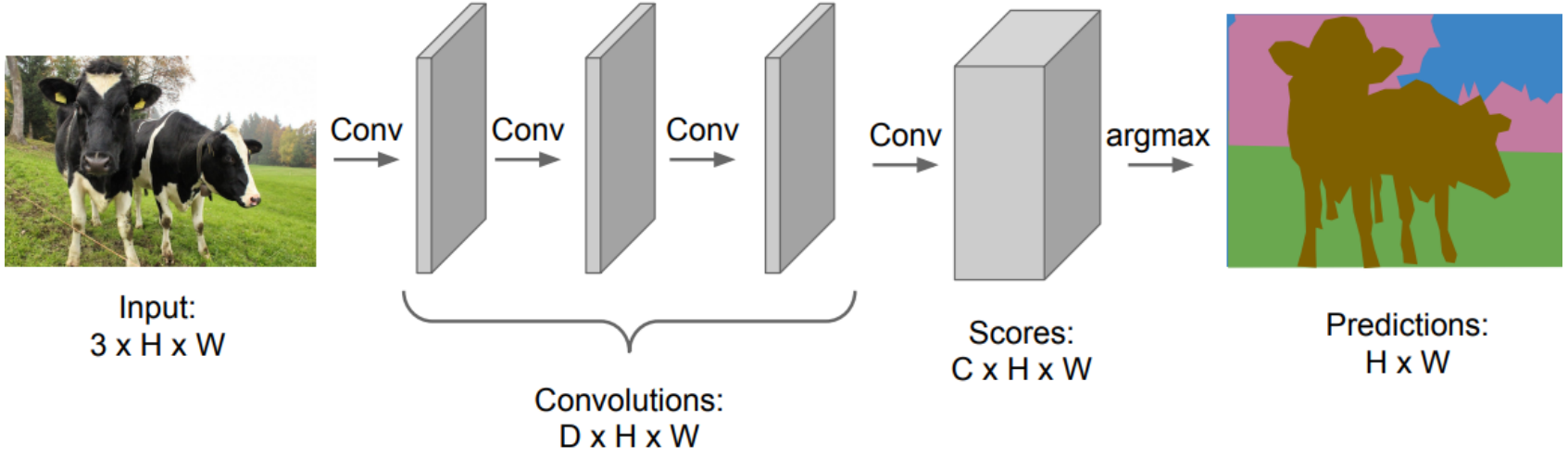
Fully Convolutional

다음 방법은 원본 이미지 사이즈를 끝까지 그대로 유지하면서 Convolution 연산을 돌리는 것임

Filter size에 맞는 적절한 stride와 padding 값을 적용하면 원본 이미지의 사이즈를 끝까지 유지할 수 있음

Ex) filter size가 3 x 3 이였다면, stride 1, padding 1로 하면 원본 사이즈를 끝까지 유지할 수 있음

하지만 이렇게 하면 padding에 의해서 의미 없는 값이 계속 부여되면서 가장자리 픽셀에 대해 결과가 안 좋을 뿐더러,
끝까지 원본 size를 유지하니까, 연산량이 너무 많아짐



어떤 Loss ?

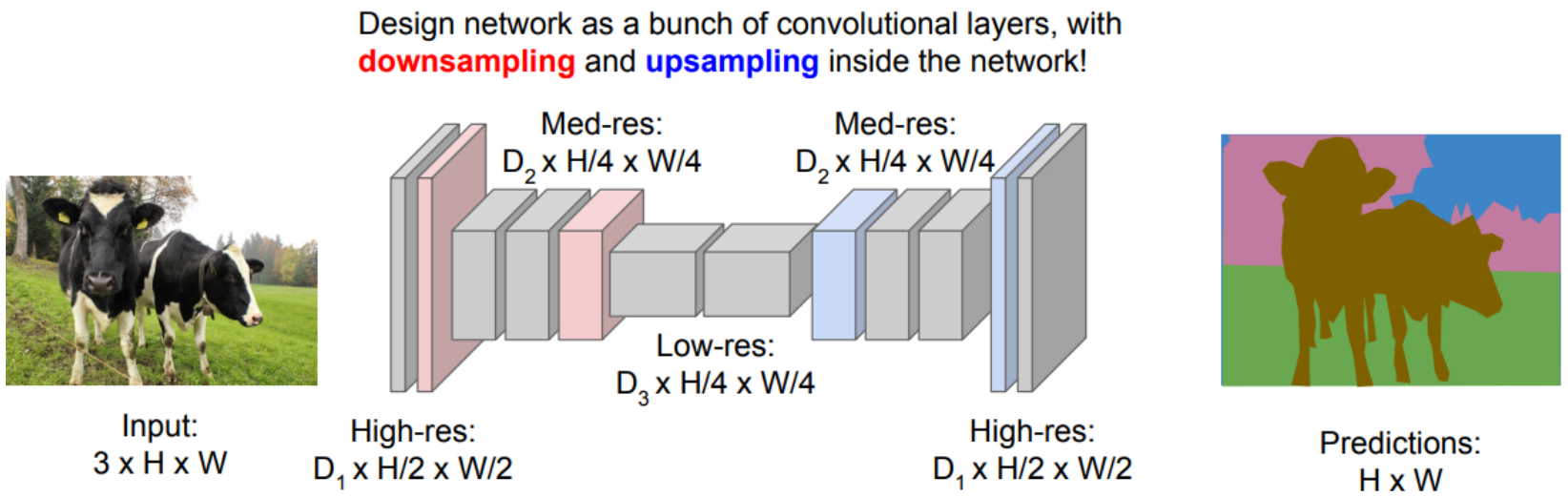
Segmentation에 사용되는 다양한 Loss가 궁금하다면 -> https://analytics4everything.tistory.com/302

Enhanced Fully Convolutional

즉, 원본을 끝까지 유지하는게 아니라 Convolution 연산으로 feature map 사이즈를 줄이고 다시 원본으로 사이즈를 복원시키는 방법이 가장 좋다는 것

일반적인 Convolution 연산은 padding을 하지 않는 이상 이미지의 사이즈가 줄어들 수 밖에 없음.

그렇지만 Upsampling이라는 연산을 통해서 줄어든 feature map에서 원본 사이즈처럼 만들어 낼 수 있음



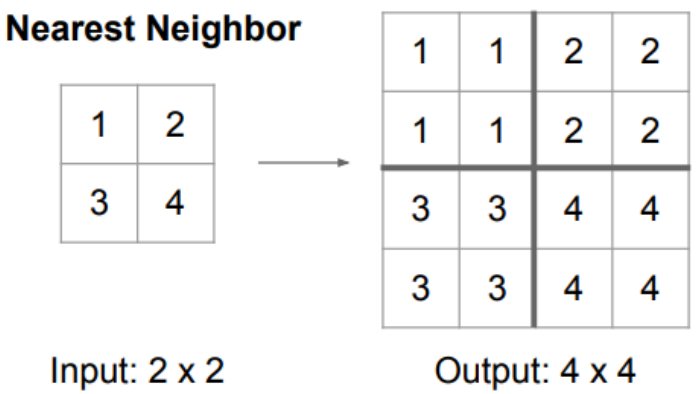
Upsampling - w/o learning

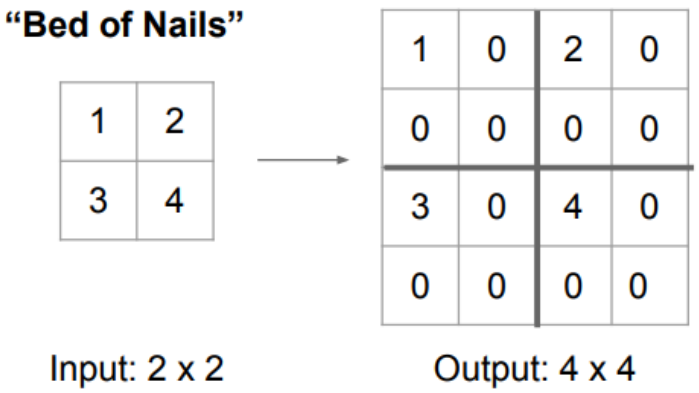
Upsampling을 하는 방법 또한 다양하게 있음, 우선은 학습이 필요 없는 방법 먼저 간단하게 확인하면 다음과 같음

Nearest Neighbor 방식은 output size를 정해두고, 각 픽셀마다 가장 가까운 input 픽셀을 보고 그 값으로 모두 치환해 넣는 방법임

Bed of Nails 방식은 input 값들은 그대로 보존하되, 나머지는 padding처럼 0으로 채워넣겠다는 것임

-> 물론 각 픽셀 주변 값을 채워 넣는게 좋아보일 수 있으나, 아무런 의미 없는 이상한 값이 채워질 수 있기에 성능이 좋지 않음. (사용 X)



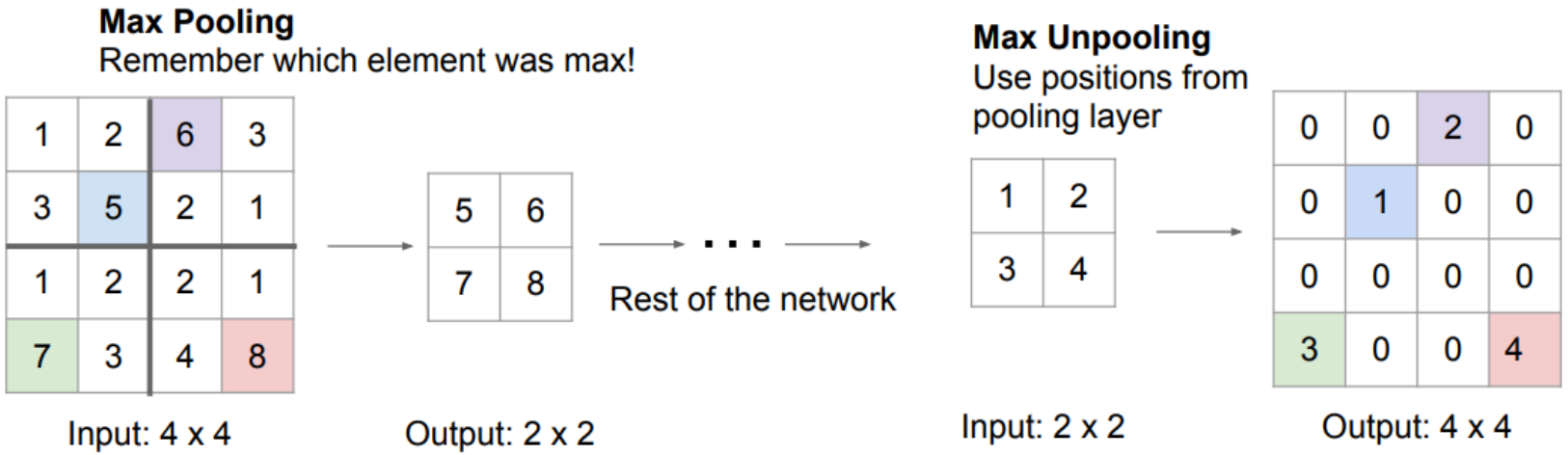


Upsampling - w/o learning

앞에서 본 방식들 말고도 몇 가지 더 있음 대표적으로 Max Unpooling이 존재함.

Bed of Nails 방식과 꽤 비슷한데, 앞에서 Max Pooling이 사용되었다면 그 위치를 기억했다가 다시 그 자리에 넣는 방식임

나머지 칸들은 Bed of Nails와 마찬가지로 0으로 치환하여 Upsampling을 수행함

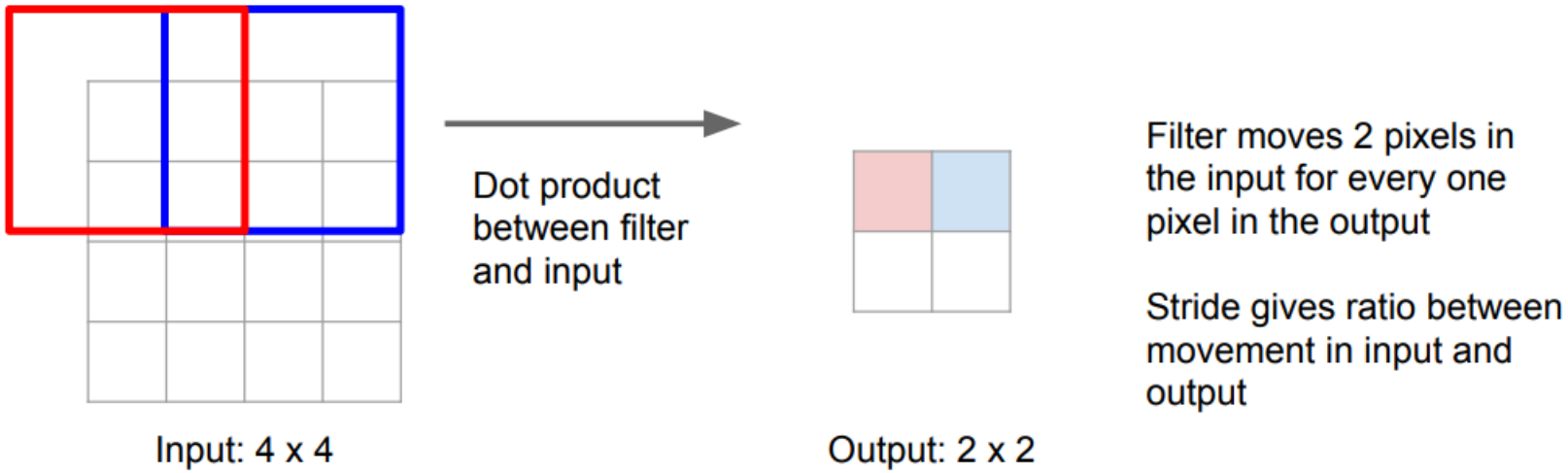


Transpose Convolution (Learnable Upsampling)

일반적인 Convolution 연산을 먼저 확인해보면, 원본 이미지에 Filter를 끼워서 Weighted Summation을 통해 하나의 값을 생성해냄

따라서 일반적인 Convolution은 padding을 적절히 적용하지 않으면 무조건적으로 사이즈를 줄이는 연산이 되는 것

Recall: Normal 3 x 3 convolution, stride 2 pad 1



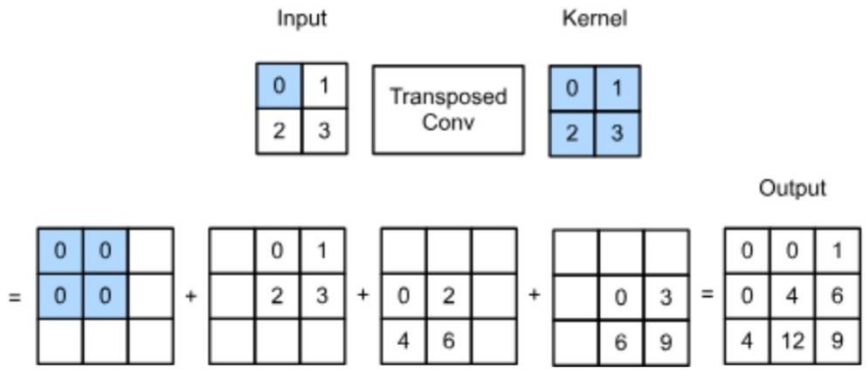
Transpose Convolution (Learnable Upsampling)

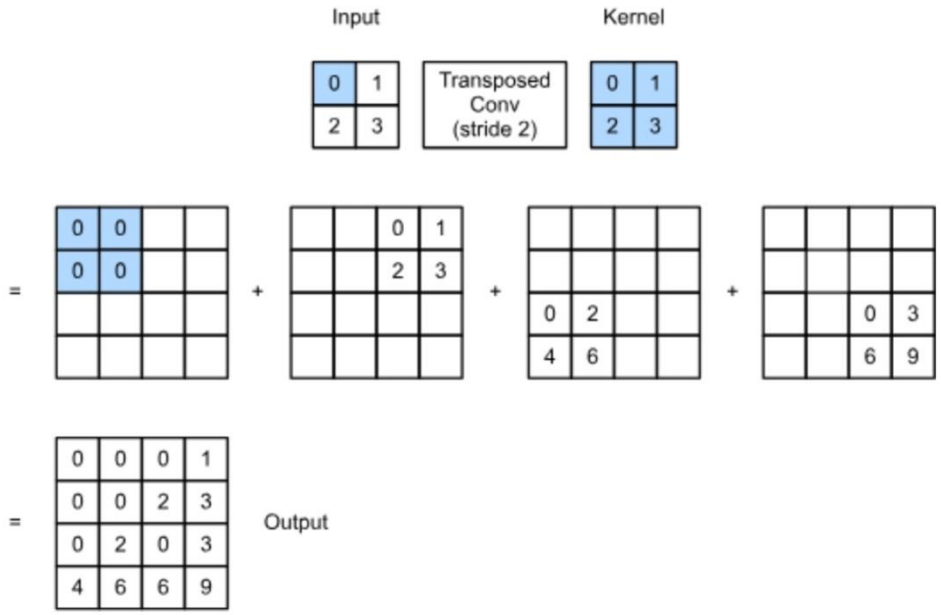
하지만 아래와 같이 Transpose Convolution을 적용하면 모델이 Upsampling을 하는 Filter 속 값들을 학습하게 됨

즉 이전 슬라이드에서 봤던 방식들과 다르게, 1이라는 하나의 값을 어떻게 하면 훨씬 더 의미있게 키울 수 있을까를 학습하는 것임

Transpose Convolution에도 stride와 같은 테크닉은 적용될 수 있음. 실제 연산은 아래 예시를 보면 이해할 수 있음

일반적인 Convolution 에서의 filter 역할) 특정 Feature를 추출하기 위해서 어떻게 생긴 filter를 구성해야 할까 ?
Transpose Convolution 에서의 역할) 어떻게 하면 특정 Feature를 잘 보존하면서 size를 키울 수 있을까 ?

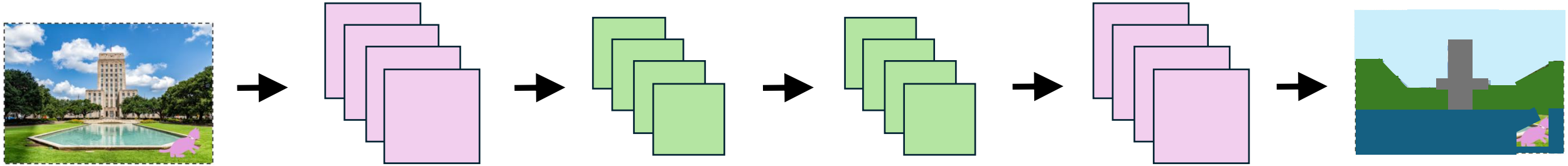




Transpose Convolution (Learnable Upsampling)

전체 형태를 쉽게 보면, 아래와 같음

Pooling을 거치며 Feature map size를 줄인 다음 거기서 바로 Deconvolution (Transpose Convolution)을 거쳐서 원본 size와 동일하게 만들어줌



Auto Encoder

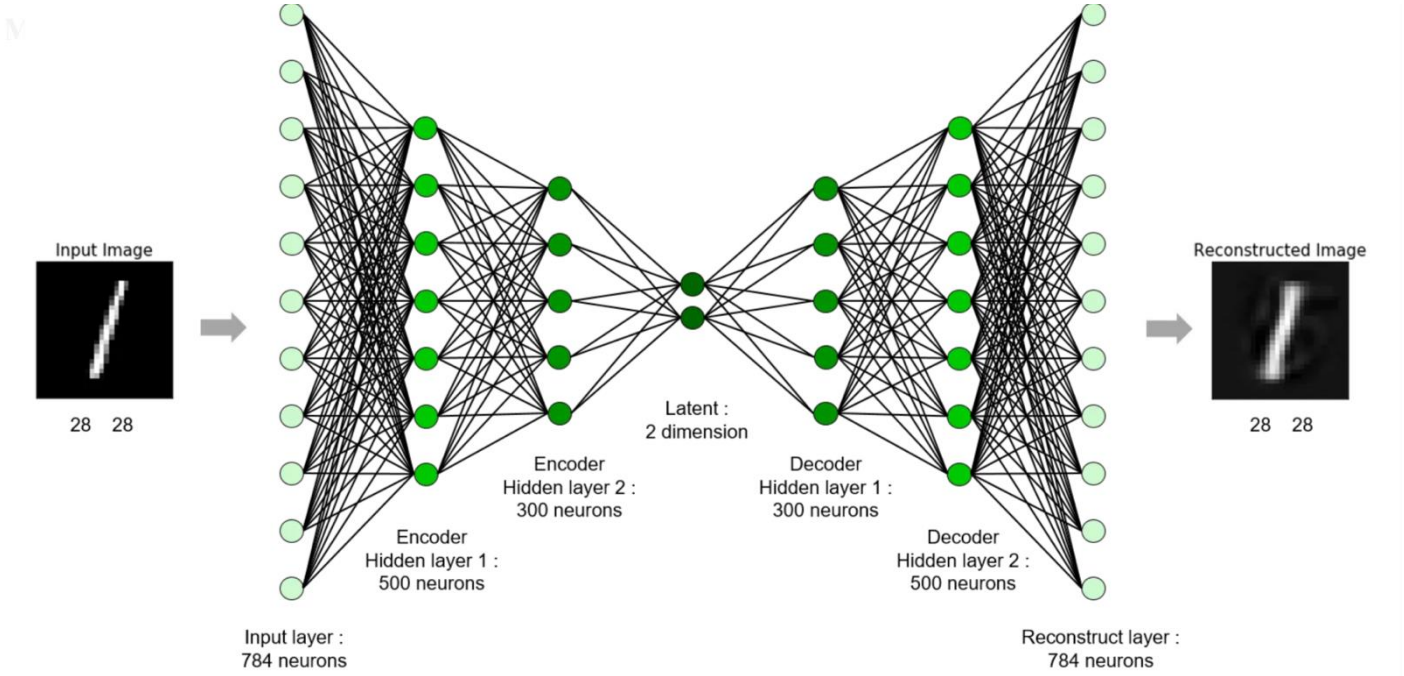
AutoEncoder

결국 input과 동일한 사이즈로 output을 만들어낼 수 있다는 것임

즉, output이 뭔가 특별한 결과(bbox의 위치, 객체의 class 등)가 아니라 원본과 비슷한 녀석을 만들어낼 수 있음

AutoEncoder는 input을 output과 동일하게 두고 모델을 학습시키는 녀석이라서 Ground Truth에 해당하는 Y 값이 바로 Input 자기 자신임

따라서 모델은 자기 자신을 잘 Encoding 했다가 다시 복원하는 가중치들을 학습하게 되는 것임



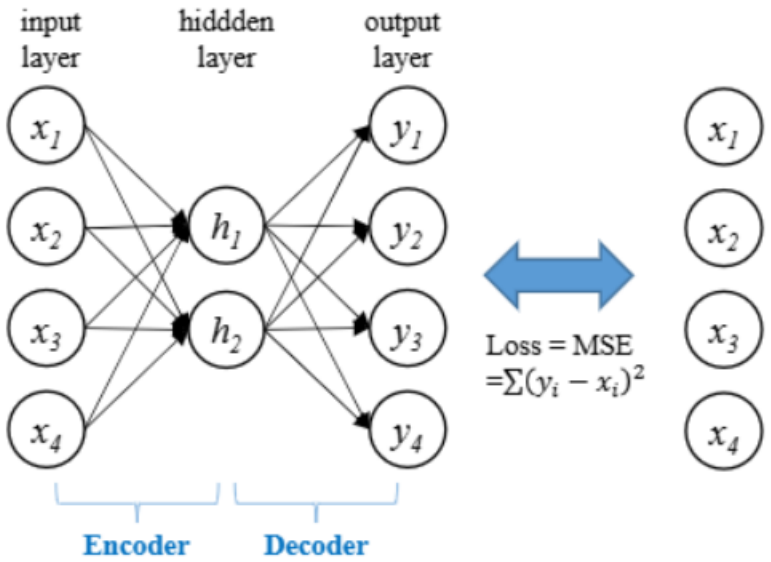
AutoEncoder

이런 AutoEncoder는 어디에 쓰이냐면, 이상치 탐지 Anomaly Detection에 주로 사용됨

왜냐면 가지고 있는 데이터들이 일반적으로 가지는 경향을 바탕으로 복원하는 가중치들을 학습하기 때문에,

새로운 데이터가 일반적인 경향을 가지지 못하고, 이상한 경우에는 복원을 제대로 하지 못함

이러한 특성을 반영해서 If (Loss(Y, X) < threshold) -> X는 정상치 / Else X는 이상치 이렇게 모델을 설계할 수 있는 것임

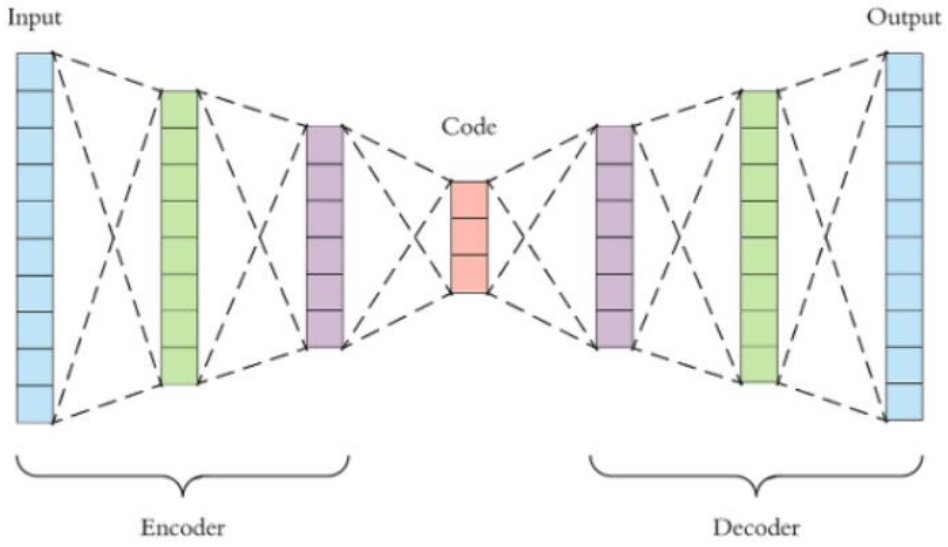


Enhancing AutoEncoder

AutoEncoder의 성능을 높이기 위해서 몇 가지 방법들이 등장했음

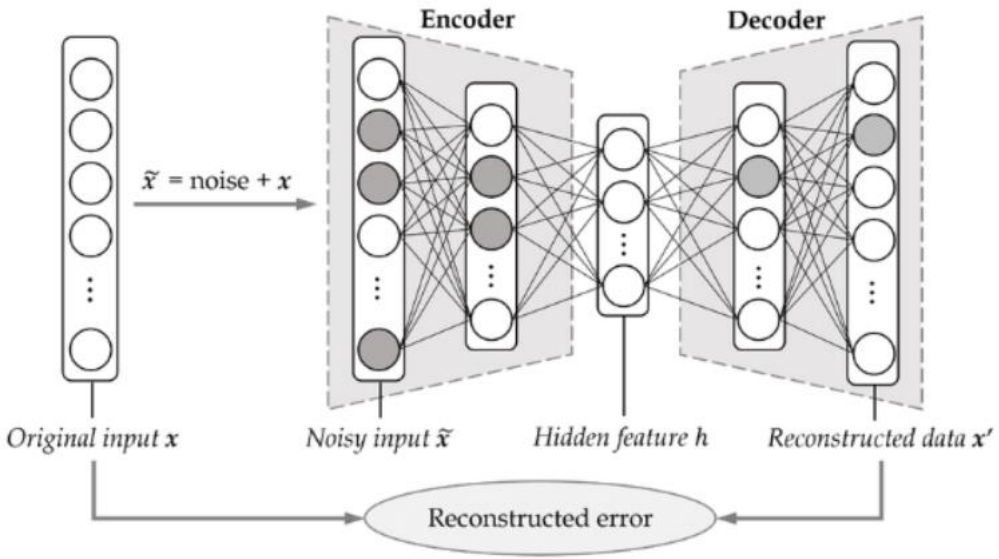
Stacked Autoencoder

엄청난 테크닉이 들어간건 아니고, 조금 더 깊은 은닉층을 추가해서
학습을 강화시킨 모형임



Denoising Autoencoder

일부러 원본에 noise를 섞어서 원본가 조금 다른 Input을 넣지만
Loss 계산할 때는 원본 Input을 사용함
-> 훨씬 더 어려운 조건에서 원본을 복원해야 하니 모델 강화

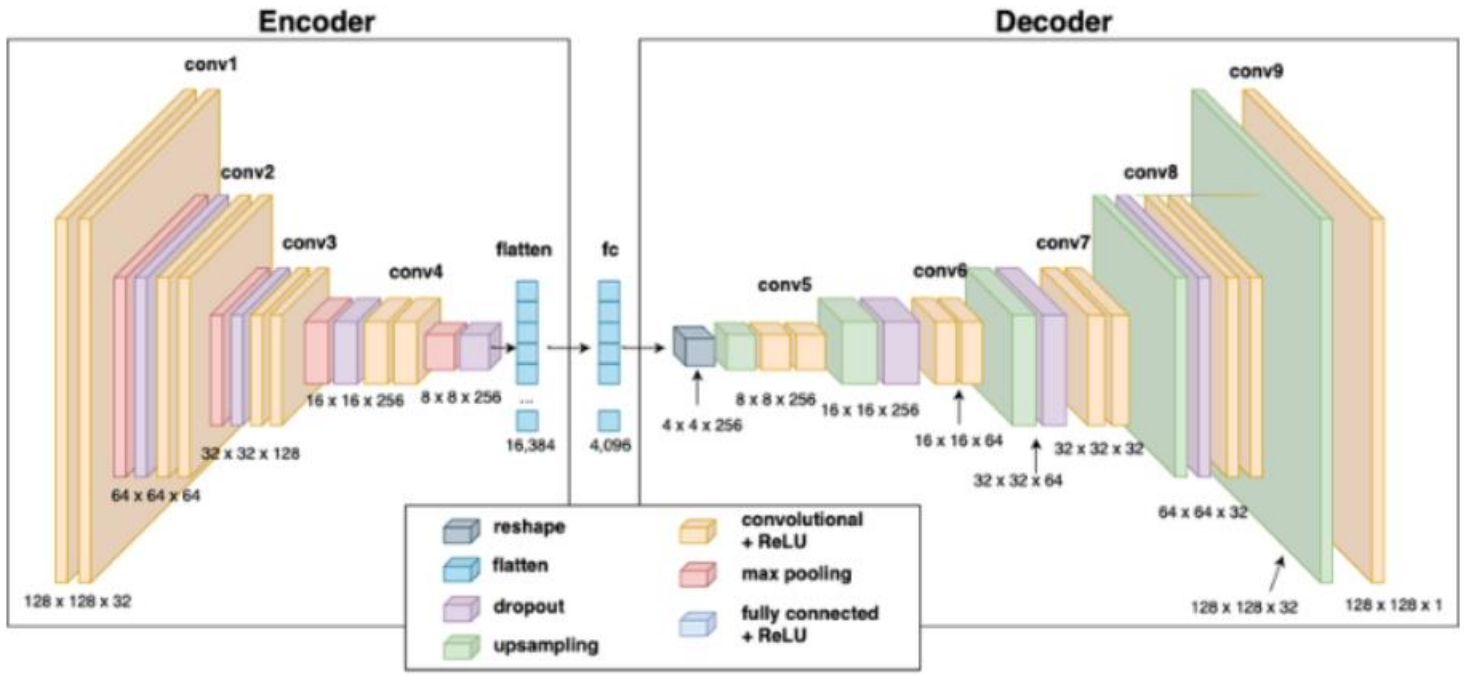


Convolutional AutoEncoder

이미지에서 AutoEncoder의 성능을 높일 수 있도록, Convolution이 추가되었을 뿐임. (Segmentation과 비슷)

Encoder에서 Convolution 연산을 통해서 데이터를 특정 Vector로 압축시키고

압축된 Vector를 기반으로 원본 이미지를 돌려내는 Decoder를 연결 (Deconvolution)



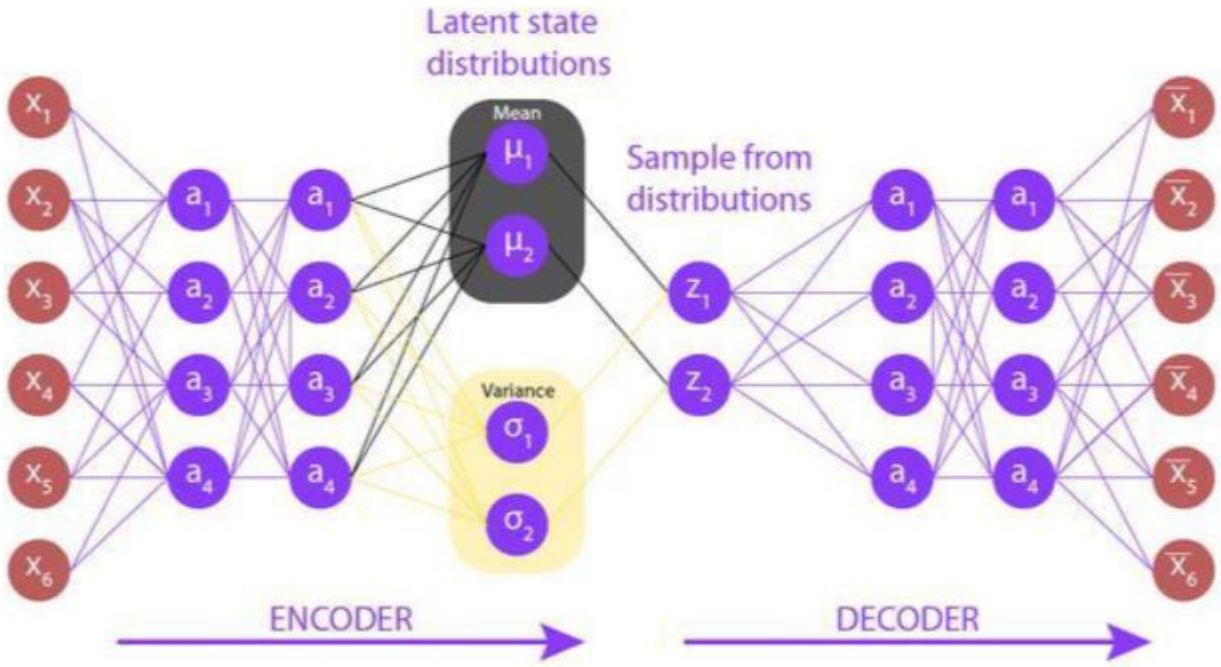
Variational AutoEncoder

앞에서 본 AutoEncoder들은 결국 이미지나 무언가를 생성하긴 하는데, 항상 똑같은 모습으로 생성하게 됨

하지만 Variational AutoEncoder는 복잡한 데이터셋에서 확률 분포를 학습하게 됨

즉, 고양이는 보통 이런 분포를 따르더라를 알게 되는 것임

그러고 나서 이 모델을 사용할 때는 새로운 고양이 사진을 넣는게 아니라, 분포에서 시작 지점만 골라주면 그 분포일 때의 이미지가 생성되는 것임



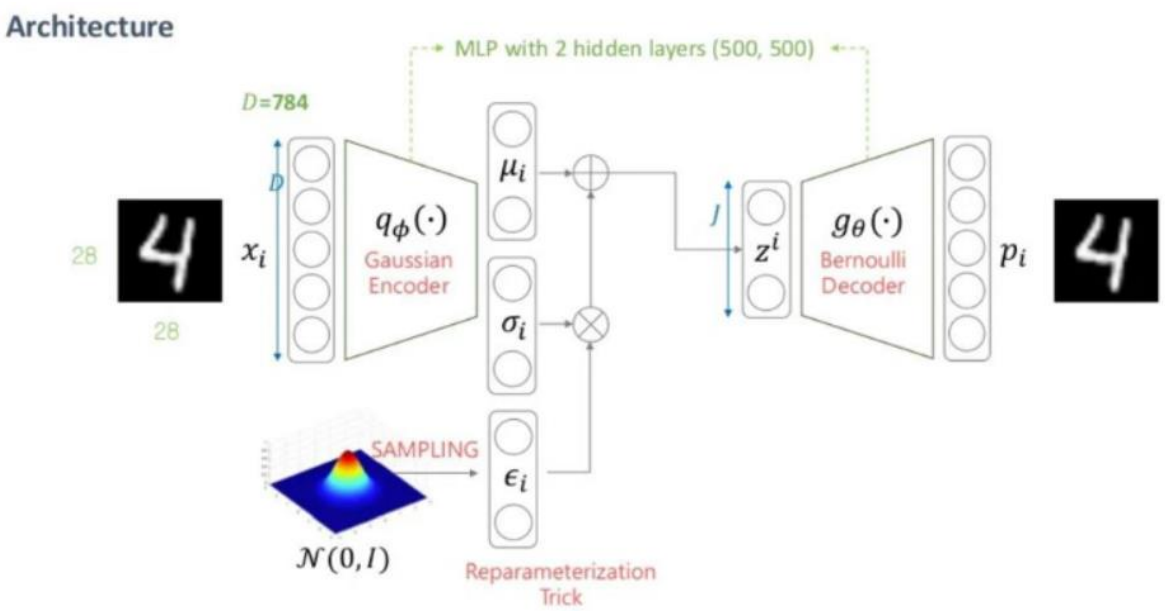
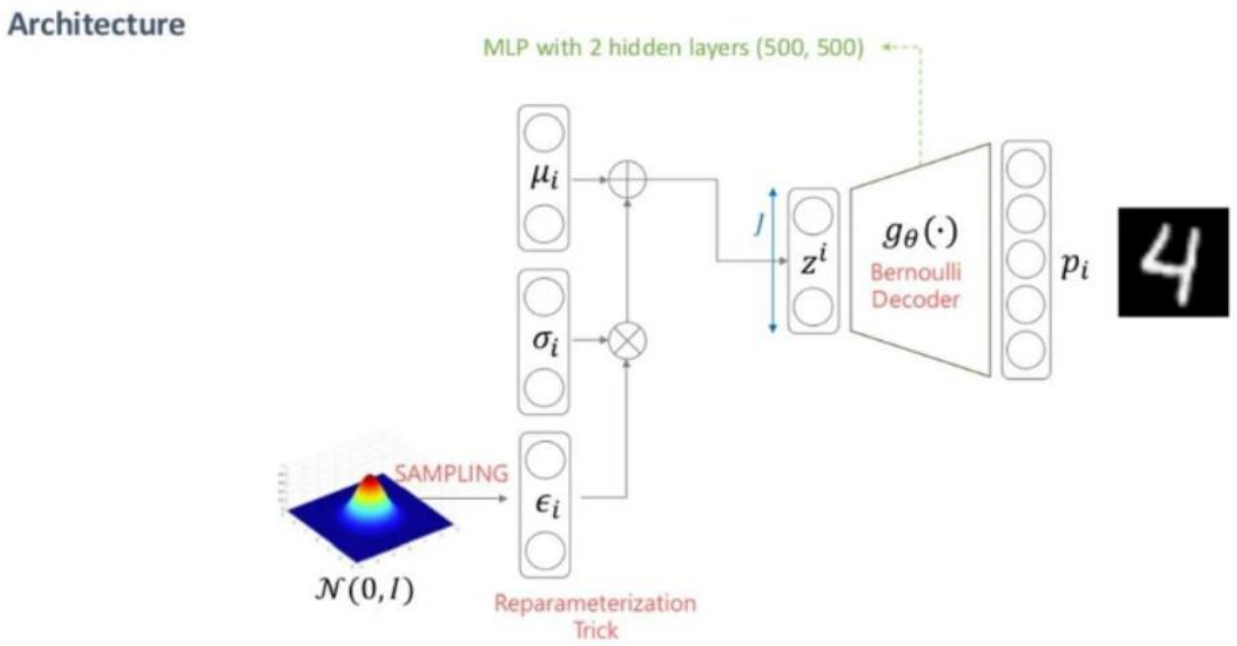
Variational AutoEncoder

즉 아래와 같이 사용하면 이것을 생성형 모델로 쓸 수 있다는 것임

대신 분포를 잘 학습해야 비슷하거나 의미 있는 이미지가 생성될 것임

이후 GAN이나 Diffusion 같은 훨씬 더 고차원적인 생성형 모델들이 개발되기 시작함

학습 시



Inference 할 때